TRAVAUX PRATIQUES • DATA VISUALISATION

VÉLIB' MÉTROPOLE

De l'open data à MySQL, puis à Tableau Desktop

COMPTE RENDU INTÉGRAL — TP2

Réalisation	Informations
Auteur	Romain GARDAIS
Formation	Data Analyst — Travaux pratiques décisionnels
Période	23–25 octobre 2026
Livrables analysés	Classeur Tableau .twbx, scripts Python, jeux JSON, GeoJSON, supports PDF, captures de travail et script SQL MySQL

Rapport fondé sur les fichiers transmis, les captures d'écran de réalisation et les éléments de la conversation de suivi « Définir un anti-consultant ». Les valeurs sont celles du jeu de données fourni, et non une mesure actuelle du réseau.

Sommaire

1. Contexte, objectifs et périmètre

2. Sources, architecture et préparation

3. Résultats et indicateurs du jeu de données

4. Réalisation des exercices 1 à 12

5. Tableaux de bord et restitution

6. Contrôles, difficultés et limites

6.1 Contrôles SQL de la couverture temporelle

Le script contient plusieurs diagnostics fondés sur COUNT(DISTINCT stationcode), COUNT(DISTINCT releve_at), MIN(releve_at), MAX(releve_at), DATE(releve_at) et HOUR(releve_at). Ces requêtes servent à distinguer le nombre de lignes, le nombre de stations couvertes et le nombre réel d'horodatages disponibles. Les captures Tableau fournissent les résultats observés ; le fichier .sql, à lui seul, ne prouve pas que chaque instruction a été exécutée avec succès.

Certains tests SQL ciblent les heures 08 h/18 h ou 09 h/19 h, alors que le diagnostic final de la collecte du 23/10/2026 utilise 12 h/15 h. Ces filtres horaires sont des essais de contrôle successifs et ne doivent pas être interprétés comme des relevés effectivement présents à toutes ces heures. Pour l'exercice 12, la densité et la régularité des observations priment sur le simple nombre de lignes.

7. Bilan et livrables

Annexes : calculs Tableau, inventaire des feuilles, sources et script SQL complet

Repérage : les intitulés des feuilles correspondent au classeur Tableau joint ; les illustrations proviennent des captures de l'utilisateur. Les captures ont été sélectionnées pour éviter les répétitions.

1. Contexte, objectifs et périmètre

Le TP vise à construire une chaîne décisionnelle complète à partir du flux public « Vélib – Vélos et bornes – Disponibilité temps réel » : acquisition JSON, transformation et stockage MySQL, exploitation dans Tableau Desktop, analyses géographiques et statistiques, tableaux de bord interactifs et examen des limites d'une prévision temporelle.

Le sujet principal prévoit 12 exercices Tableau, puis trois tableaux de bord et une histoire finale. Deux supports complémentaires approfondissent le diagramme de Pareto et le clustering k-means. La réalisation a évolué au-delà du minimum avec plusieurs variantes, contrôles et un quatrième tableau de bord de conclusion.

1.1 Objectifs pédagogiques

- Lire une API open data et un document JSON imbriqué ; maîtriser les attributs de stations et leur horodatage.
- Importer et actualiser des données relationnelles avec Python et MySQL ; constituer une table d'historique.
- Construire des mesures calculées, indicateurs KPI, cartes et interactions Tableau.
- Employer LOD FIXED, calculs de table, Pareto, statistiques descriptives et clustering.
- Présenter les résultats de manière lisible et évaluer honnêtement la qualité des données.

1.2 Sources effectivement examinées

Élément	Utilisation
TP-Velib-Open-data-MySQL-et-Tableau.pdf	Consignes générales, 12 exercices, tableaux de bord
TP-Pareto-Capacite-Velib-par-commune.pdf	Rang, cumul, axe double et seuil de 80 %
Tutoriel-Clustering-k-means-des-stations-Velib.pdf	Variables, variantes de k, lecture des clusters
velib.json / velib-disponibilite-en-temps-reel.json	Relevé de 1 519 stations du 23/10/2026
arrondissements.geojson	Fond et géométrie géographique complémentaires
test_mysql.py / import_velib.py / collecte_historique.py	Connexion, import, conservation de relevés
TP2_GARDAIS_ROMAIN(4).twbx	Feuilles, paramètres, calculs et tableaux de bord enregistrés
Captures transmises	Illustrations et traces de configuration / validation
Requete SQL TP2 GARDAIS Romain.sql	DDL MySQL, requêtes de vérification, historisation, unicité et diagnostics temporels

2. Sources, architecture et préparation

2.1 Chaîne de traitement

Source Ville de Paris (API Opendatasoft) → export JSON UTF-8 → script Python de normalisation → MySQL « velibdb » (station_dispo) → copie horodatée dans station_historique → Tableau Desktop → feuilles, paramètres, cartes, analyses et tableaux de bord.

La table station_dispo représente l'état le plus récent importé. La table station_historique conserve des instantanés datés ; ces deux tables ne doivent pas être confondues lors des agrégations, sous peine de dupliquer les stations.

2.2 Acquisition et structure du JSON

Le sujet préconise l'export complet Opendatasoft plutôt que la lecture paginée. Les attributs exploités comprennent stationcode, name, capacity, numbikesavailable, numdocksavailable, mechanical, ebike, is_installed, is_renting, is_returning, duedate, coordonnées géographiques, commune et code INSEE. Les coordonnées sont initialement imbriquées dans coordonnees_geo.

2.3 Importation dans MySQL

Le script import_velib(1).py lit velib.json, demande le mot de passe MySQL sans l'inscrire dans le fichier, convertit duedate ISO 8601 en date MySQL et extrait lat/lon et les champs communaux. Il effectue un INSERT … ON

DUPLICATE KEY UPDATE dans station_dispo, ce qui permet de réactualiser une station identifiée par stationcode sans la créer plusieurs fois.

test_mysql(1).py vérifie l'ouverture de la connexion locale (port 3306) et énumère les tables. collecte_historique(1).py enregistre, avec un horodatage Europe/Paris, un instantané de station_dispo dans station_historique. Cette collecte est une opération ponctuelle par exécution du script ; l'automatisation périodique n'est pas démontrée par les seuls fichiers transmis.

```
INSERT INTO station_historique (releve_at, stationcode, capacity, numdocksavailable, numbikesavailable,
mechanical, ebike, is_renting)
SELECT %s, stationcode, capacity, numdocksavailable, numbikesavailable, mechanical, ebike, is_renting
FROM station_dispo;
```

2.3.1 Script SQL : création, contrôle et historisation

Le fichier « Requete SQL TP2 GARDAIS Romain.sql » complète les scripts Python en documentant la structure de la base velibdb et les requêtes de vérification. Les instructions sont reproduites intégralement dans l'annexe D ; les extraits ci-dessous expliquent les opérations principales.

Schéma relationnel et granularité

station_dispo possède une ligne par station et une clé primaire stationcode (VARCHAR(10)). station_historique possède une clé primaire technique id (BIGINT AUTO_INCREMENT) et enregistre un relevé par station et par horodatage releve_at (DATETIME). Les index idx_releve et idx_station facilitent les recherches temporelles et par station. Le script ne déclare pas de clé étrangère entre les deux tables.

Contrôle du volume et de la présence des tables

```
SELECT 'station_dispo' AS nom_table, COUNT(*) AS nombre_lignes
FROM station_dispo
UNION ALL
SELECT 'station_historique' AS nom_table, COUNT(*) AS nombre_lignes
FROM station_historique;
```

Cette requête vérifie les volumes des deux tables après chargement. SHOW TABLES et un SELECT ... LIMIT 20 servent à confirmer leur présence et à inspecter les colonnes importées.

Unicité des observations historiques

```
SELECT stationcode, releve_at, COUNT(*) AS nombre
FROM station_historique
GROUP BY stationcode, releve_at
HAVING COUNT(*) > 1;
```

La requête détecte plusieurs lignes partageant le même couple (stationcode, releve_at). Le script ajoute ensuite une contrainte UNIQUE sur ce couple. Cette contrainte doit être créée après suppression d'éventuels doublons ; l'ALTER TABLE peut échouer si des doublons existent déjà.

Deux modes distincts de datation de l'historique

Dans le fichier SQL, l'instruction INSERT INTO station_historique ... SELECT duedate ... FROM station_dispo utilise la date duedate fournie par la source pour renseigner releve_at. À l'inverse, le script Python collecte_historique.py utilise un horodatage de collecte. Il s'agit de deux sémantiques différentes : dernière mise à jour de la station d'un côté, moment de la photographie du réseau de l'autre. Il faut choisir et documenter l'une d'elles pour une série temporelle cohérente.

La contrainte UNIQUE (stationcode, releve_at) empêche les doublons mais peut également provoquer une erreur lors d'une nouvelle insertion des mêmes couples. L'insertion SQL fournie n'emploie ni IGNORE ni ON DUPLICATE KEY UPDATE : elle ne garantit donc pas à elle seule une réexécution sans erreur.

2.4 Préparation de la source Tableau

Le classeur utilise la source « station_dispo (velibdb) », avec les champs de station_dispo et station_historique, et une source géographique complémentaire « arrondissements.geojson ». Les champs stationcode et code_insee sont traités comme identifiants, non comme grandeurs additives. Les mesures lat/lon sont utilisées pour les cartes.

Point de vigilance : le PDF évoque une vue v_station et un utilisateur MySQL tableau ; les scripts remis se connectent à velibdb avec root. Ce rapport décrit l'implémentation effectivement observable sans supposer la présence de toutes les vues ou tous les comptes du corrigé.

3. Résultats et indicateurs du jeu de données

Indicateur	Valeur calculée	Lecture
Stations distinctes	1 519	Volume du réseau dans le JSON fourni
Vélos disponibles	18 707	Somme numbikesavailable
Électriques	7 335	39,2 % des vélos disponibles
Mécaniques	11 372	Complément des électriques
Capacité totale	49 896	Places théoriques du réseau
Capacité moyenne	32,85	Places par station
Capacité médiane	30	Places par station
Stations situées à Paris	996	65,6 % des stations
Stations sans vélo disponible	113	À distinguer du statut métier « Vide »
Stations de capacité nulle	3	Exclues des taux divisant par capacity

Les captures Tableau affichent notamment 1 519 stations, 18 707 vélos et 39 % électriques : ces trois KPI concordent avec les deux fichiers JSON. Le PDF complémentaire Pareto utilise un relevé antérieur et indique 49 893 places ; le JSON remis totalise 49 896 places. Cet écart de trois places est lié à des instantanés différents et ne doit pas être effacé.

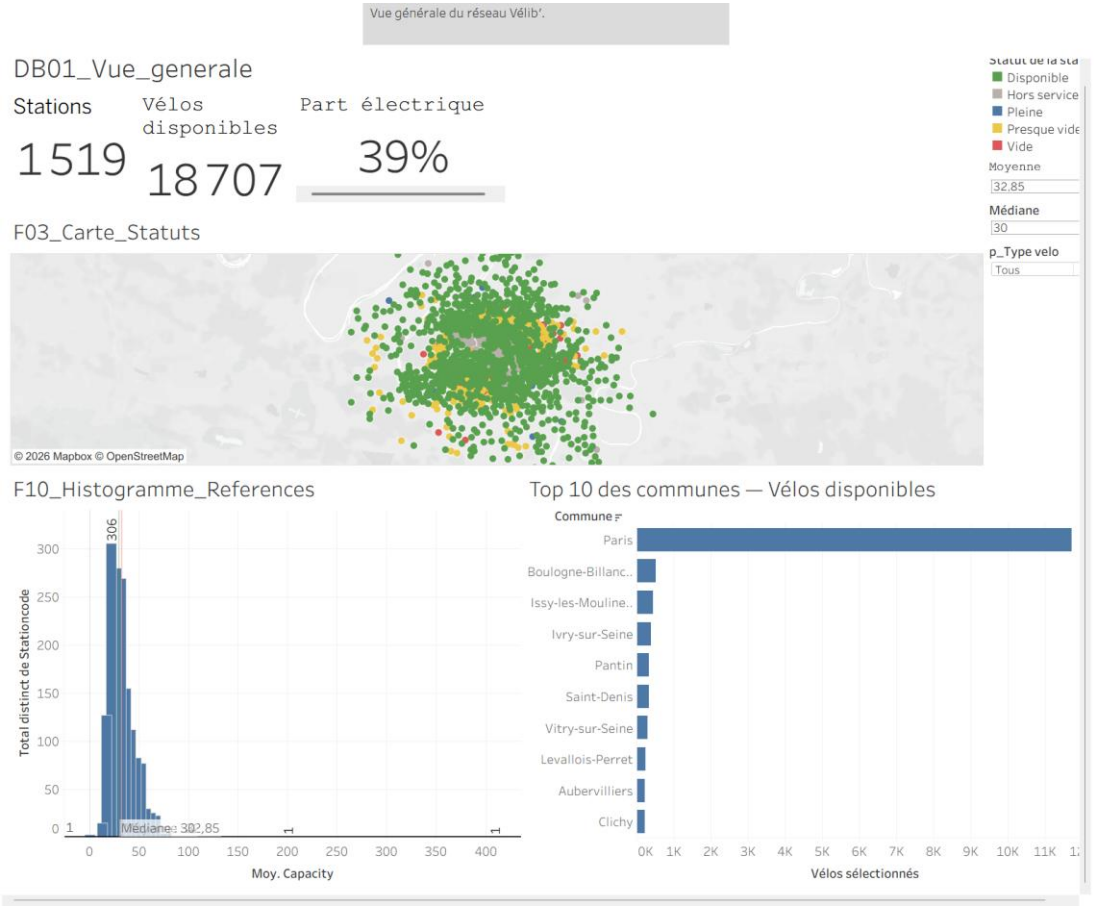


Figure — Tableau de bord général : KPI, carte des statuts, histogramme et classement communal.

4. Réalisation des exercices 1 à 12

Exercice 1 — Connexion MySQL et préparation de la source

Objectif : rendre le flux exploitable dans Tableau et maîtriser les dimensions, mesures et types géographiques. La présence de la source « station_dispo (velibdb) » dans le classeur et des champs issus de l'historique confirme l'exploitation des données. Les scripts permettent de reconstituer le parcours d'import et de contrôle de connexion.

Justification méthodologique : la relation logique entre une table d'état et une table d'historique préserve le niveau station lorsqu'elle est correctement définie, contrairement à une jointure physique qui peut multiplier les lignes d'une station par le nombre de relevés historiques. Une connexion directe privilégie l'actualité des données ; un extrait privilégie souvent la stabilité, la portabilité et certaines fonctions statistiques.

Exercice 2 — Premières visualisations

Les feuilles F02_Stations_par_commune, F02_Capacite_departement et F02_Controle_Stations permettent de compter les stations distinctes, comparer les capacités et vérifier le grain des données. Les classements par commune rendent visible la forte concentration des stations à Paris (996 sur 1 519 dans le JSON).

L'emploi de COUNTD([Stationcode]) est essentiel : COUNT ou SUM d'un compteur de lignes peut être trompeur lorsque des relevés historiques sont associés à une même station.

Exercice 3 — Champs calculés et qualité

Le classeur contient les champs Statut de la station, Arrondissement, Fraîcheur (min), Donnée fiable ?, taux_remplissage, Part électriques et Taux station. Les vues F03_Statuts, F03_Carte_Statuts, F03_Controle_Fraicheur et F03_Stations_Figees rendent ces indicateurs contrôlables.

```
// Fraîcheur d'un relevé (minutes)
DATEDIFF('minute', [duedate], NOW())
// Statut : station non louable, vide, pleine, presque vide, disponible
IF [is_installed] = "NON" OR [is_renting] = "NON" THEN "Hors service"
ELSEIF [numbikesavailable] = 0 THEN "Vide"
ELSEIF [numdocksavailable] = 0 THEN "Pleine"
ELSEIF [numbikesavailable] <= 2 THEN "Presque vide"
ELSE "Disponible" END
```

Le contrôle de fraîcheur est nécessaire : un enregistrement géolocalisé peut être présent mais ancien. Les stations de capacité nulle et les dates atypiques doivent être isolées des interprétations opérationnelles.

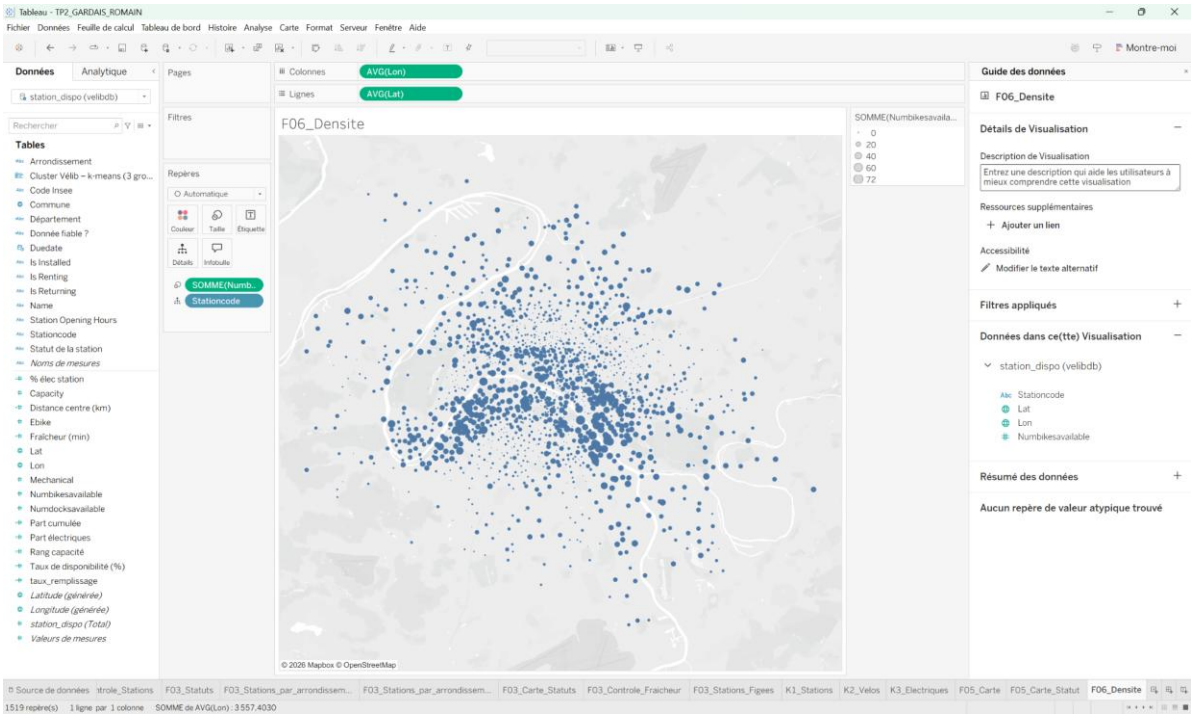


Figure — Carte Tableau des stations : distribution géographique et statut.

Exercice 4 — Indicateurs clés

Les feuilles K1_Stations, K2_Velos et K3_Electriques présentent respectivement 1 519 stations, 18 707 vélos disponibles et 39 % de vélos électriques. Leur agrégation dans DB01_Vue_generale permet une lecture immédiate avant les analyses détaillées.

Exercice 5 — Cartographie des stations

Les feuilles F05_Carte et F05_Carte_Statut utilisent Longitude/Latitude et Stationcode sur Détail, afin de conserver un point par station. La taille des cercles représente une quantité (notamment la capacité), la couleur un statut ou un taux. Des filtres Département, Commune et Statut facilitent l'exploration de la petite couronne et de Paris.

Interprétation : les zones denses de stations sont principalement situées dans Paris ; la taille des symboles ne doit pas être confondue avec leur nombre. Une carte par statut révèle des stations vides, presque vides ou indisponibles sans prouver, à elle seule, la cause de ces situations.

Exercice 6 — Analyse spatiale avancée

Les feuilles F06_Densite, F06_Rayon, F06_Taux_distance, F06_Electriques_2km et F06_Station_plus_eloignee étudient les distances, la concentration spatiale et l'accessibilité. Le champ Distance centre (km) repose sur MAKEPOINT et DISTANCE, avec le centre de Paris (48,8566 ; 2,3522). Le paramètre p_Rayon km permet de modifier le périmètre analysé.

```
DISTANCE(MAKEPOINT([Lat], [Lon]), MAKEPOINT(48.8566, 2.3522), 'km')
```

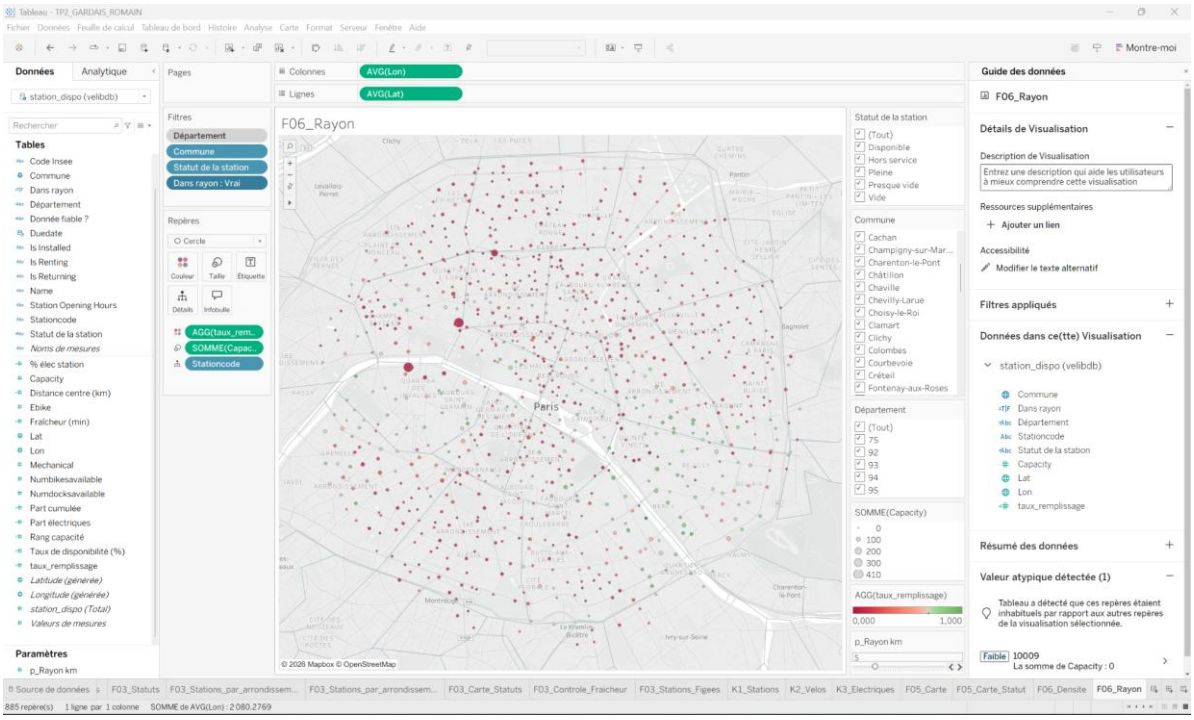


Figure — Analyse spatiale : filtrage et visualisation d'un sous-ensemble de stations.

Exercice 7 — Info-bulle enrichie

F07_Info_bulle présente une carte de stations avec une info-bulle contenant le nom, le code station, les coordonnées, la capacité, les vélos électriques et mécaniques, les bornettes libres, le statut et une jauge Unicode de disponibilité. La feuille F07_Graphique_info_bulle représente les deux catégories de vélos sous forme de barres.

Le fichier Tableau contient effectivement une insertion Viz in Tooltip : <Sheet name="F07_Graphique_info_bulle" ... filter="<All Fields>">. L'intégration est donc vérifiable dans le classeur. En revanche, la transmission correcte de la station survolée vers les champs historiques doit être testée en situation : la présence de la balise ne garantit pas, seule, la pertinence des valeurs historiques pour chaque station.

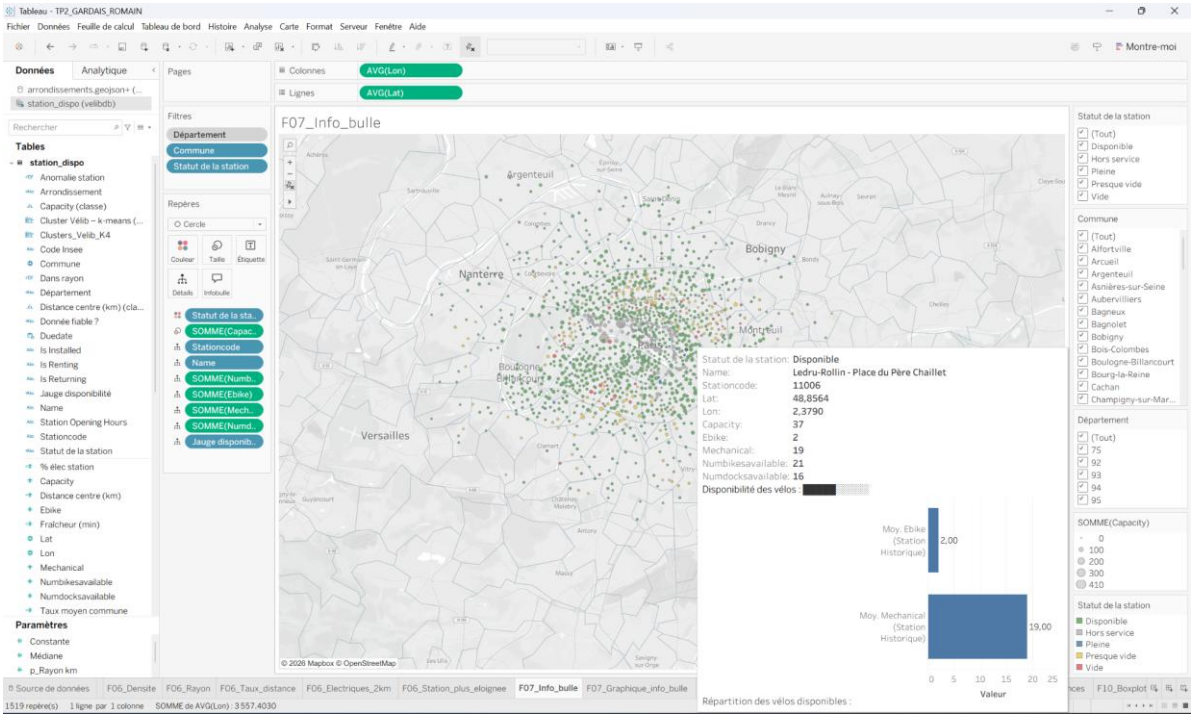


Figure — Capture de l'info-bulle enrichie avec mini-visualisation intégrée.

Exercice 8 — Paramètres et interactions

Trois paramètres interactifs sont visibles dans le classeur : p_Type velo (Tous / Mécanique / Électrique), p_Seuil critique (valeur initiale 2) et p_Top N (valeur initiale 10). Le champ Vélos sélectionnés utilise CASE pour adapter la mesure au type de vélo choisi. Les feuilles F08_Carte_Type_velo, F08_Stations_critiques et F08_Top_N_stations illustrent leur application.

```
CASE [p_Type velo]
WHEN "Mécanique" THEN [Mechanical]
WHEN "Électrique" THEN [Ebike]
ELSE [Numbikesavailable]
END
```

La carte des stations critiques filtre celles dont la mesure choisie est inférieure ou égale au seuil paramétrable. Le Top N classe les stations par SUM(Vélos sélectionnés) et limite le nombre affiché au paramètre p_Top N ; un tri décroissant améliore la comparaison.

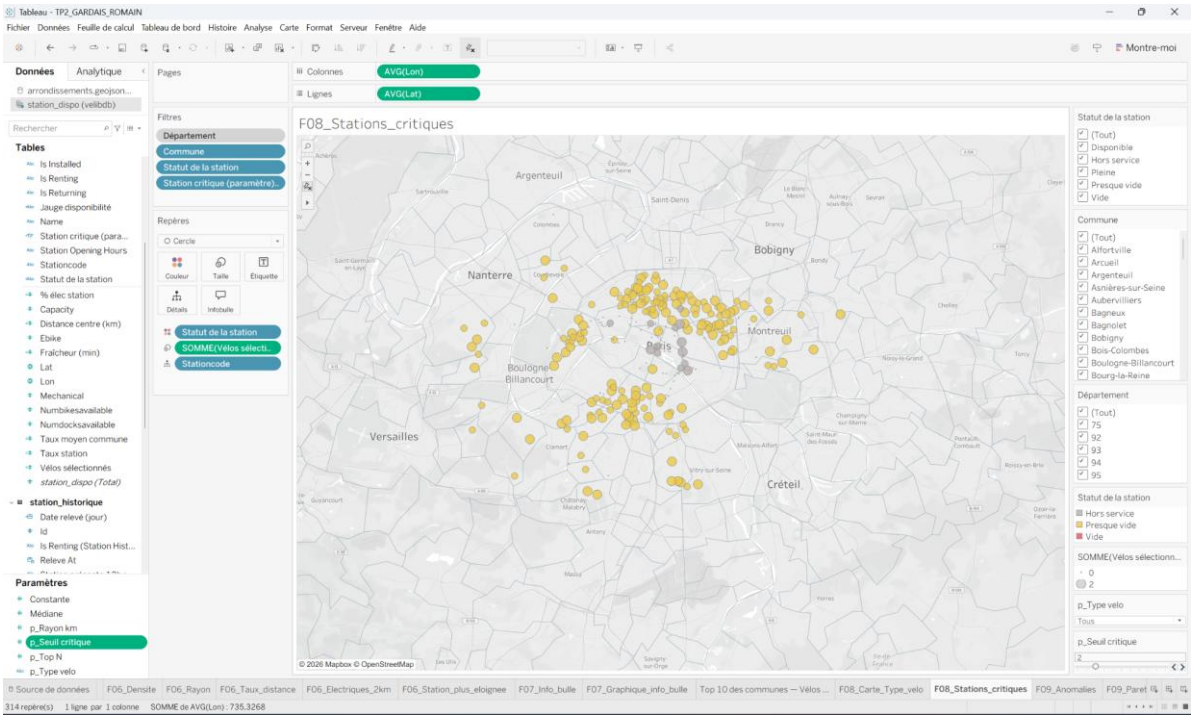


Figure — Stations critiques : résultat de l'application d'un seuil de disponibilité.

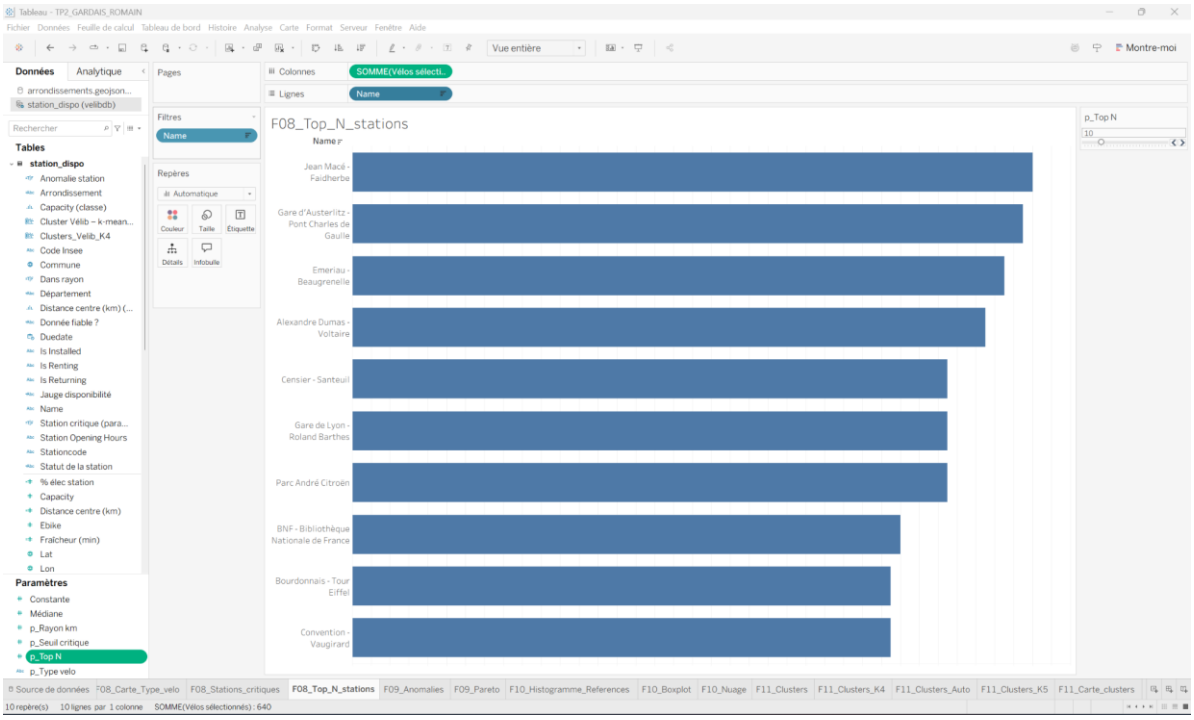


Figure — Top N dynamique des stations, trié selon le nombre de vélos sélectionnés.

Exercice 9 — LOD, anomalies et Pareto

Le calcul FIXED au niveau de la commune fournit un taux de référence stable ; l'écart station permet de repérer les stations plus ou moins remplies que leur commune. Les feuilles F09_Anomalies et F09_Controle_Pareto complètent F09_Pareto. Les calculs de table RANK, RUNNING_SUM et TOTAL produisent le classement et la part cumulée des capacités.

```
// Taux moyen de disponibilité par commune
{ FIXED [Commune] : SUM([Numbikesavailable]) / SUM([Capacity]) }
// Part cumulée de capacité (calcul de table)
RUNNING_SUM(SUM([Capacity])) / TOTAL(SUM([Capacity]))
```

Le support Pareto fourni indique que, sur son relevé du 15/10/2026, Paris concentrait 69,6 % des places et que la 10e commune faisait franchir le seuil de 80 % (80,7 %). Ces valeurs sont celles du support, pas nécessairement du relevé JSON du 23/10/2026. Le graphique associe barres de capacité, courbe cumulée et ligne de référence à 80 %.

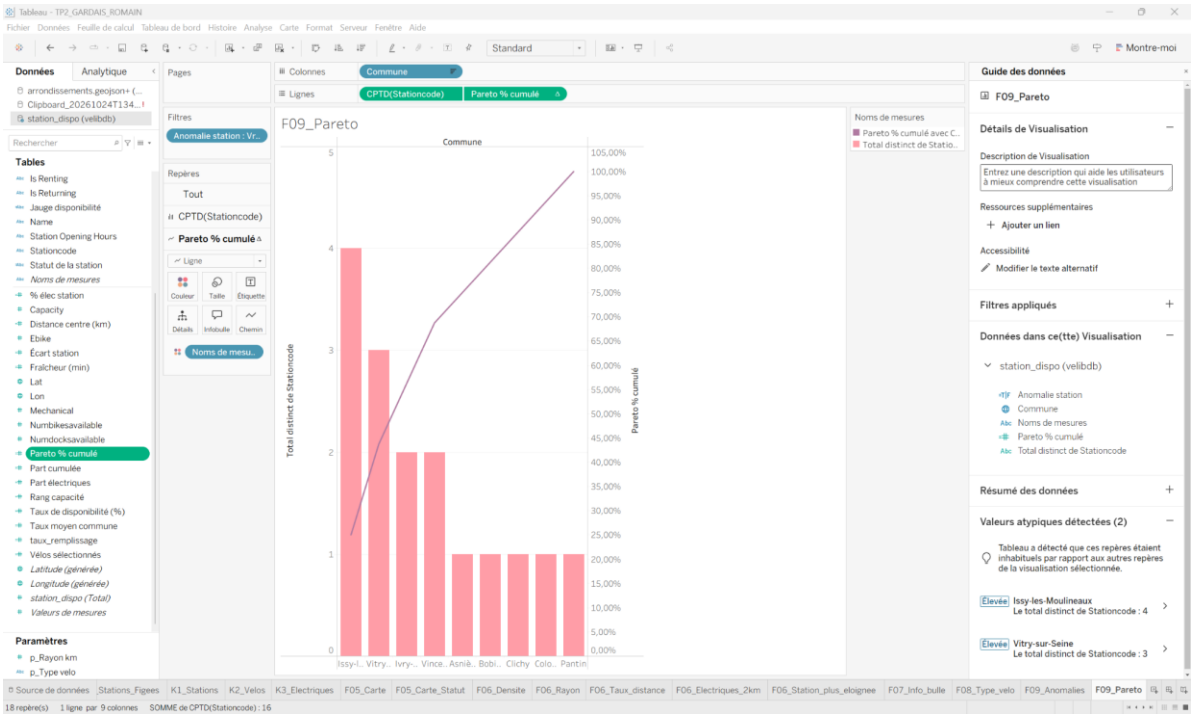


Figure — Diagramme de Pareto construit dans Tableau : capacité et cumul.

Exercice 10 — Statistiques descriptives

Les feuilles F10_Histogramme_References, F10_Boxplot et F10_Nuage abordent trois lectures complémentaires : distribution des capacités, dispersion des taux par département et relation entre capacité et vélos disponibles. L'histogramme peut afficher les repères moyenne 32,85 et médiane 30, cohérents avec le JSON transmis. Le nuage de points inclut une ligne de tendance ; la dispersion ne permet pas, sans examen de la régression, de conclure à une causalité.

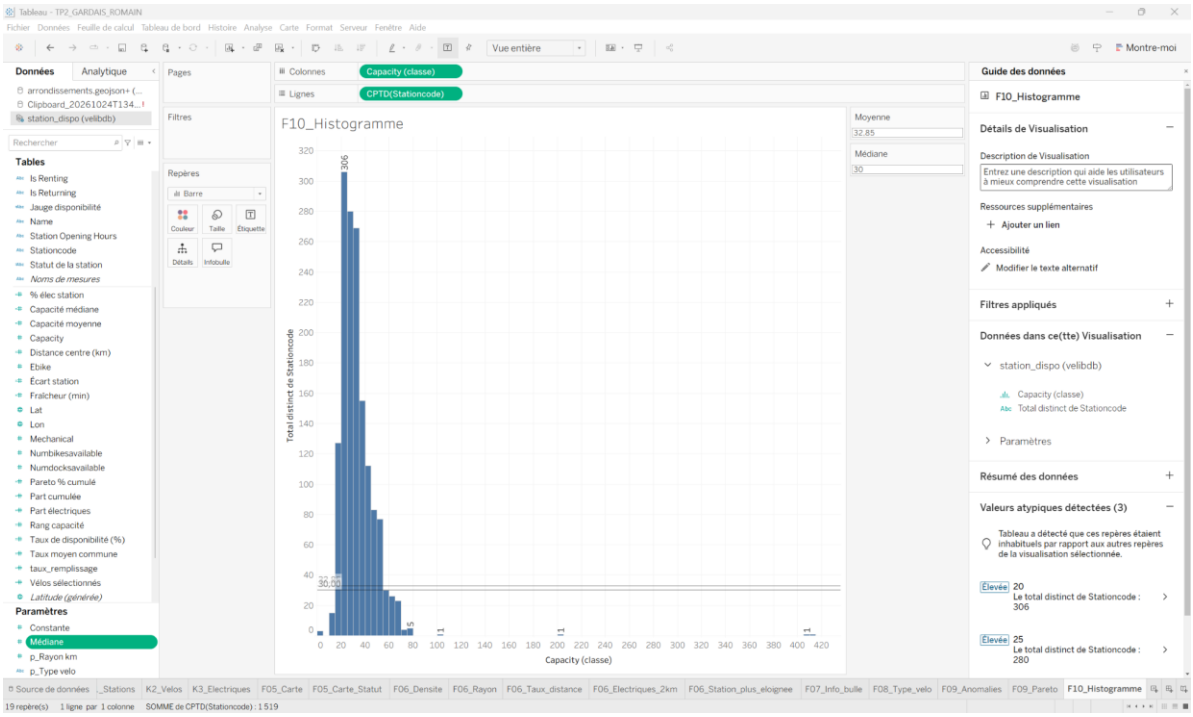


Figure — Histogramme des capacités avec classes et repères statistiques.

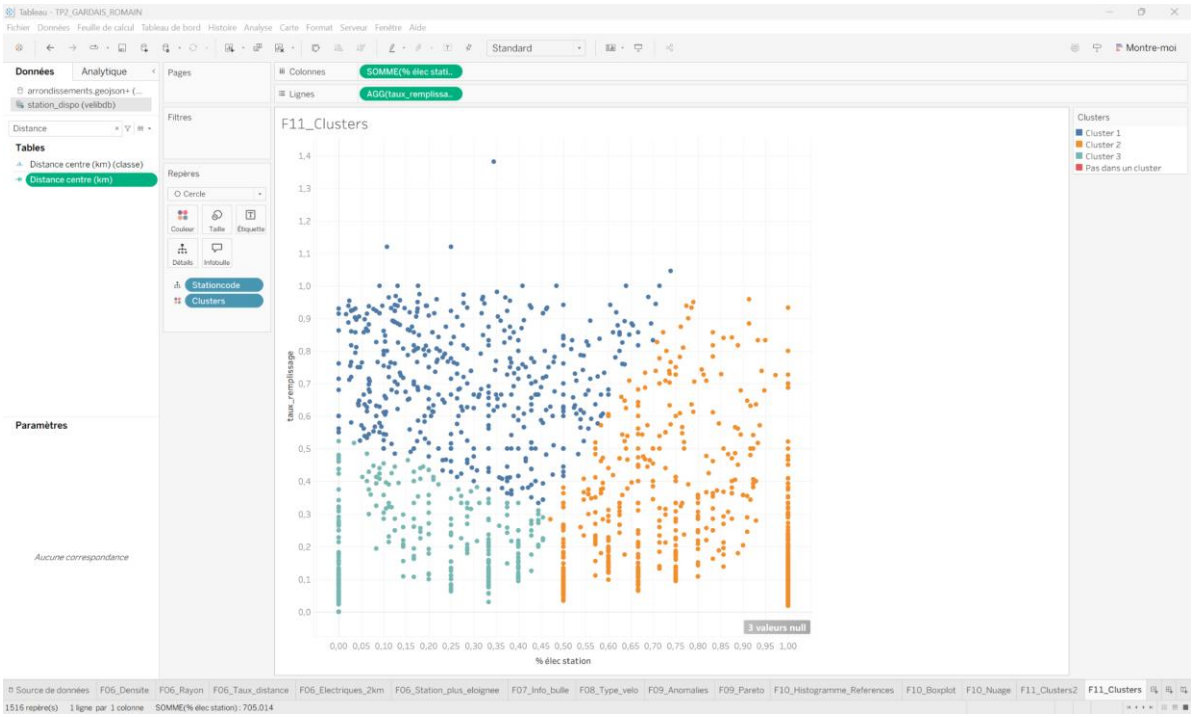


Figure — Nuage de points des caractéristiques de stations et exploration de leur relation.

Exercice 11 — Segmentation par k-means

La segmentation cherche des profils homogènes à partir du taux de remplissage, de la proportion électrique, de la capacité et de la distance au centre. Le classeur contient F11_Clusters, F11_Clusters_Auto, F11_Clusters_K4, F11_Clusters_K5 et F11_Carte_clusters, ainsi qu'un groupe enregistré Clusters_Velib_K4. Ces variantes montrent que plusieurs nombres de groupes ont été testés.

Le support précise que les variables sont mises à l'échelle avant le calcul et recommande de comparer la séparation des groupes (Between/Total), les centres et les variables discriminantes. Le nombre de groupes et les noms métier ne doivent pas être inférés d'une couleur seule. Trois stations à capacité nulle posent une difficulté pour le taux de remplissage.

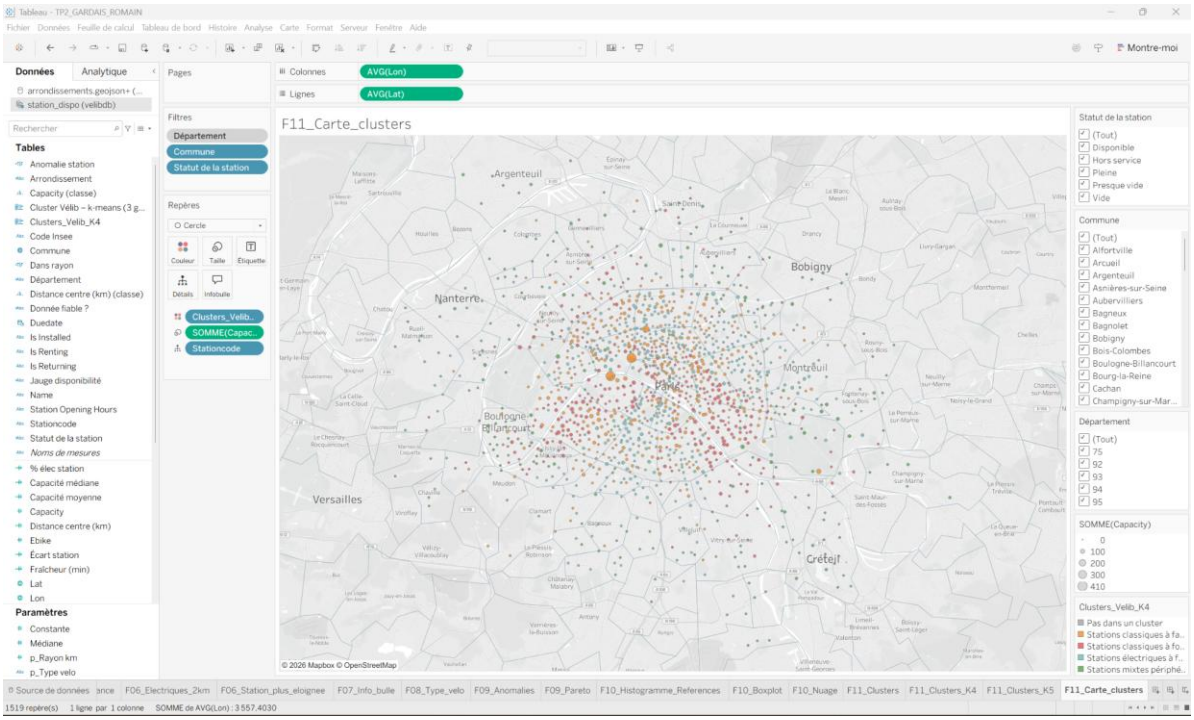


Figure — Cartographie des groupes de stations après clustering.

Exercice 12 — Historique et prévision de disponibilité

La collecte est insuffisante pour produire une prévision robuste. Les contrôles montrent 3 038 enregistrements répartis sur 18 dates, mais 3 020 se concentrent le 23/10/2026. Les captures de contrôle détaillent trois heures : 08 h (1 station), 12 h (1 500 stations) et 15 h (1 519 stations). Les observations des autres dates sont isolées ou peu nombreuses ; une courbe sur l'ensemble de la période créerait une illusion de continuité temporelle.

Relevé du 23/10/2026	Stations distinctes observées
08 h	1
12 h	1 500
15 h	1 519
Total des observations horaires	3 020

La feuille « F12 – Disponibilité moyenne des vélos à 12h et 15h » et les deux feuilles Controle_dates_historique / Controle_heures_historique sont présentes dans le .twbx. Une feuille nommée F12_Prevision_disponibilite apparaît dans les captures de la conversation mais pas dans la version .twbx jointe : elle n'a pas été retrouvée dans cet état sauvegardé. Aucun modèle de prévision Tableau exploitable ne peut donc être déclaré validé.

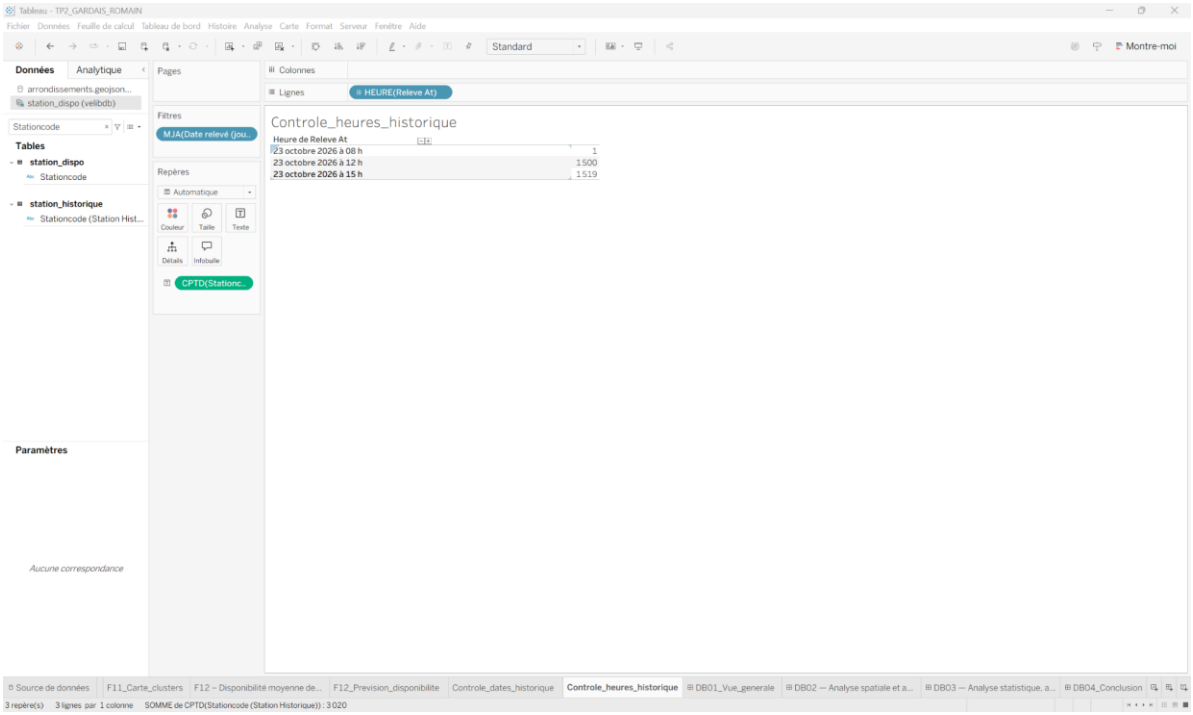


Figure — Contrôle de l'historique : 1, 1 500 et 1 519 stations selon l'heure du relevé.

Conclusion de l'exercice : la construction du jeu historique et le diagnostic sont réalisés ; il faudrait collecter plusieurs jours ou semaines de relevés à fréquence régulière, puis contrôler les trous et la stabilité du périmètre avant d'utiliser la prévision de Tableau.

5. Tableaux de bord et restitution

Le classeur joint contient six objets Tableau de type « dashboard » : DB01_Vue_generale, DB02 — Analyse spatiale et accessibilité des stations Vélib', DB03 — Analyse statistique, anomalies et segmentation, DB04_Conclusion, H01_Synthese_analyse_Velib et un tableau de bord Vélib' supplémentaire. Il contient 47 feuilles de calcul. L'objet H01 est enregistré techniquement comme un tableau de bord et non comme un objet Tableau « Story » ; la synthèse narrative existe, mais l'exigence formelle d'une histoire Tableau native reste à distinguer.

5.1 DB01 — Vue générale

Synthèse des KPI, carte de statut, histogramme et Top 10 communal. Il sert de point d'entrée et montre l'échelle du réseau et sa distribution géographique.

5.2 DB02 — Analyse spatiale

Regroupement des vues de rayon, densité, éloignement et accessibilité. Il répond aux questions « où se trouvent les stations ? » et « quelles stations sont accessibles dans un périmètre donné ? ».

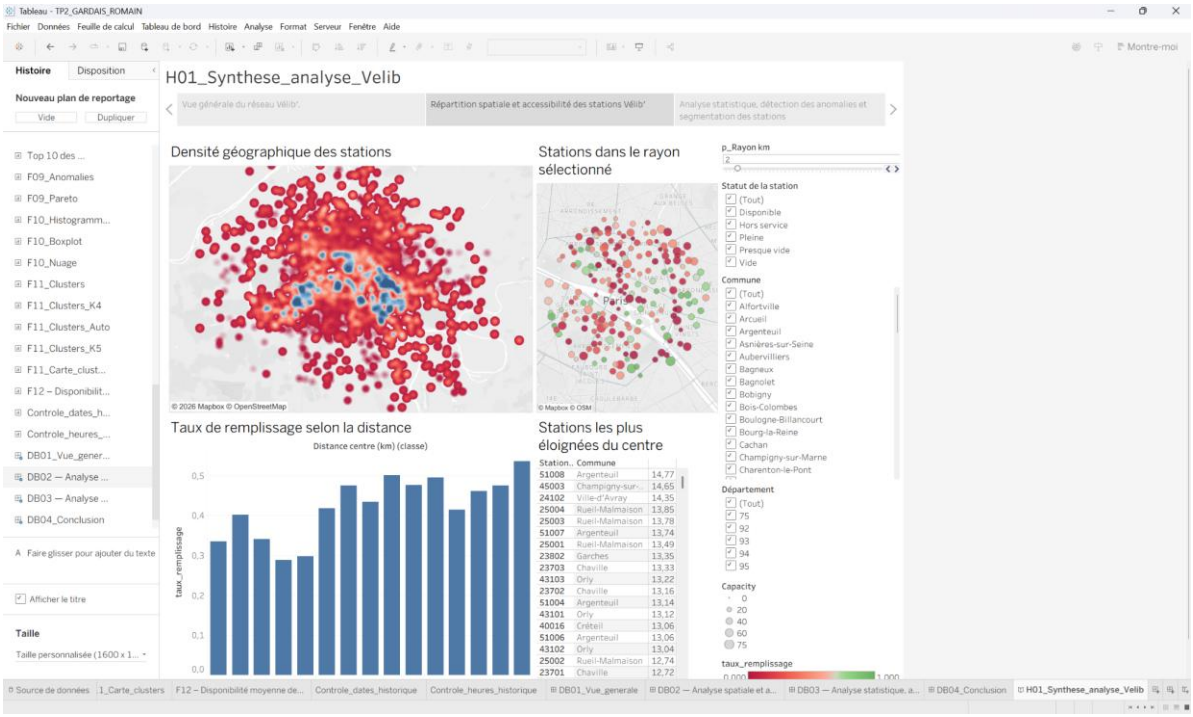


Figure — DB02 : analyse spatiale et accessibilité des stations.

5.3 DB03 — Statistiques et segmentation

Réunit histogrammes, analyses des anomalies, nuage de points, boîtes à moustaches et clustering pour passer du descriptif à la segmentation.

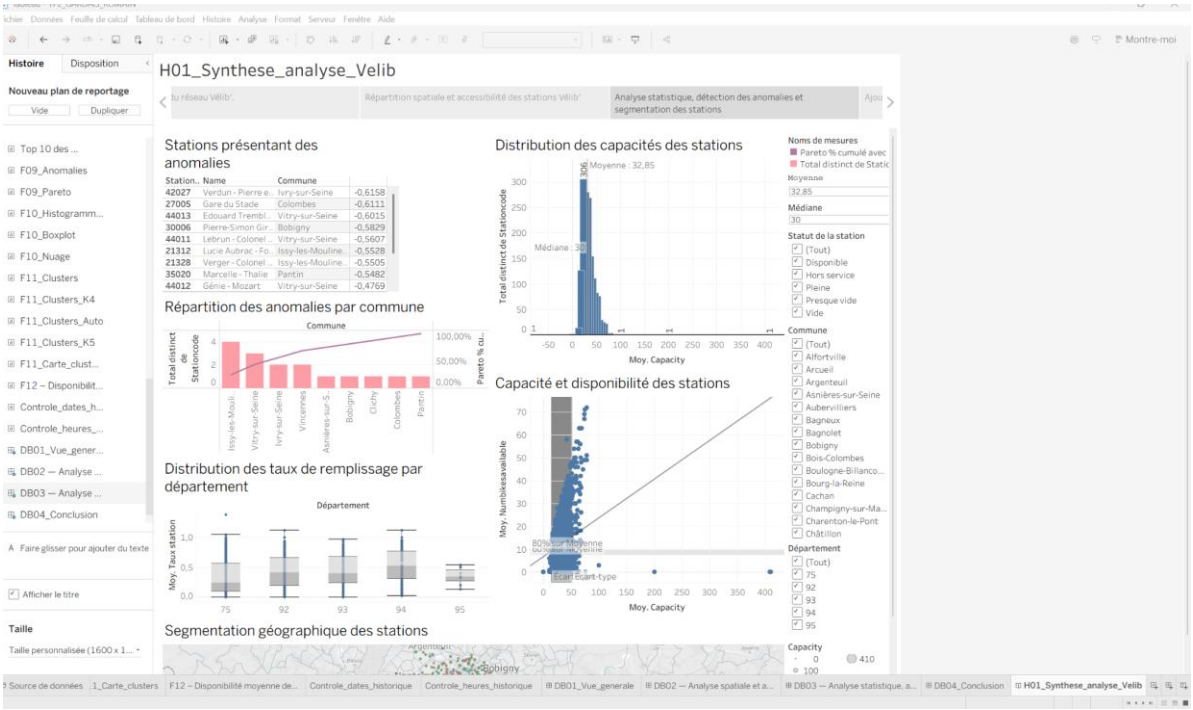


Figure — DB03 : distributions, anomalies et segmentation.

5.4 DB04 — Conclusion et synthèse H01

Le tableau de bord de conclusion reprend les enseignements essentiels, les limites de fraîcheur et les pistes d'amélioration. H01_Synthese_analyse_Velib joue le rôle de synthèse narrative dans le fichier transmis, même s'il ne s'agit pas d'un objet Story natif.

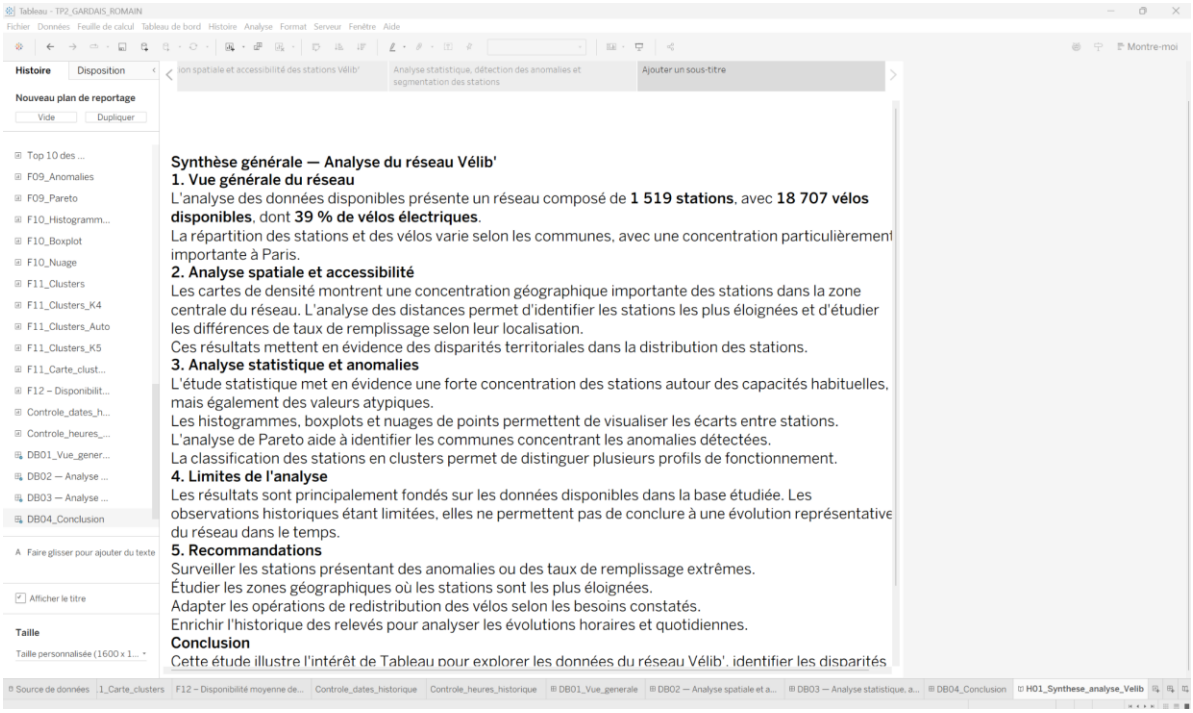


Figure — Éléments de conclusion et restitution narrative dans Tableau.

6. Contrôles, difficultés et limites

Point de contrôle	Constat	Conséquence
Grain de station	1 519 stations dans le JSON	Utiliser COUNTD(stationcode)
Capacité nulle	3 stations	Éviter les divisions par zéro
Dates de mise à jour	Dates anciennes présentes	Qualifier la fraîcheur des mesures
Historique	Relevés majoritairement concentrés sur un jour	Prévision non robuste
Sources et relevés	PDF Pareto du 15/10 vs JSON du 23/10	Ne pas fusionner les chiffres de dates différentes
Infobulle historique	Balise Viz in Tooltip présente	Tester le filtrage inter-feuilles
Objet H01	Dashboard et non Story dans le .twbx	Vérifier l'exigence de livraison formelle
Classeur empaqueté	Source et extrait inclus dans .twbx	Vérifier la réouverture et la fraîcheur des données

Plusieurs difficultés ont été traitées pendant le TP : configuration des champs et calculs, reconnaissance de dimensions temporelles, choix des agrégations, affichage des graphiques, réglage des filtres, création de paramètres, limites des analyses statistiques sur des données incomplètes. Ces corrections sont documentées par les captures successives.

7. Bilan et livrables

Le TP aboutit à une chaîne de traitement réutilisable, un classeur Tableau riche en visualisations et plusieurs tableaux de bord. Les acquis couvrent la collecte, le SQL, la préparation de données, la cartographie, les paramètres, les expressions LOD, les calculs de table, la statistique descriptive et le clustering.

Bloc	Bilan
Acquisition JSON / Python	Scripts et fichiers remis
MySQL et historique	Import, test et script de collecte présents

Exercices 1 à 11	Feuilles et calculs correspondants présents dans le classeur
Exercice 12	Analyse de faisabilité réalisée ; prévision non validée
Tableaux de bord	Quatre tableaux de bord DB01–DB04 présents, plus H01 et un dashboard additionnel
Histoire Tableau native	Non retrouvée dans le .twbx joint
Traçabilité	PDF, scripts, JSON, GeoJSON, classeur et captures disponibles

Améliorations prioritaires : (1) automatiser la collecte toutes les 15 à 60 minutes sur une période représentative ; (2) définir des contrôles de qualité et de doublons ; (3) vérifier l'actualisation des données et les filtres de l'info-bulle ; (4) créer une Story native si exigée ; (5) sauvegarder la dernière feuille de prévision exploratoire si sa présence dans le livrable est souhaitée.

Annexes — Calculs, scripts et références

A. Formules clés effectivement présentes

- Part électriques : IF SUM([numbikesavailable]) = 0 THEN NULL ELSE SUM([ebike]) / SUM([numbikesavailable]) END
- Taux station : IF [capacity] > 0 THEN ([mechanical] + [ebike]) / [capacity] END
- Station critique (paramètre) : [Vélos sélectionnés] <= [p_Seuil critique]
- Part cumulée : RUNNING_SUM(SUM([capacity])) / TOTAL(SUM([capacity]))
- Distance centre (km) : DISTANCE(MAKEPOINT([lat],[lon]), MAKEPOINT(48.8566,2.3522), "km")
- Date relevé (jour) : DATE(DATETRUNC("day", [releve_at]))

B. Inventaire abrégé des feuilles par exercice

Exercice	Feuilles identifiées
02	F02_Stations_par_commune, F02_Capacite_departement, F02_Controle_Stations
03–05	F03_Statuts, F03_Stations_Figees, F03_Carte_Statuts, F05_Carte, F05_Carte_Statut
06	F06_Densite, F06_Rayon, F06_Taux_distance, F06_Electriques_2km, F06_Station_plus_eloignee
07–08	F07_Info_bulle, F07_Graphique_info_bulle, F08_Carte_Type_velo, F08_Stations_critiques, F08_Top_N_stations
09–10	F09_Anomalies, F09_Controle_Pareto, F09_Pareto, F10_Histogramme_References, F10_Boxplot, F10_Nuage
11	F11_Clusters, F11_Clusters_Auto, F11_Clusters_K4, F11_Clusters_K5, F11_Carte_clusters
12	F12 – Disponibilité moyenne des vélos à 12h et 15h, Controle_dates_historique, Controle_heures_historique

C. Documents de référence

- Support principal : TP-Velib-Open-data-MySQL-et-Tableau(3).pdf, 39 pages.
- Support complémentaire : TP-Pareto-Capacite-Velib-par-commune(3).pdf, 4 pages.
- Support complémentaire : Tutoriel-Clustering-k-means-des-stations-Velib(3).pdf, 4 pages.
- Scripts : test_mysql(1).py, import_velib(1).py, collecte_historique(1).py.
- Données : velib.json, velib-disponibilite-en-temps-reel.json, arrondissements.geojson.
- Classeur : TP2_GARDAIS_ROMAIN(4).twbx, inspecté comme archive Tableau (structure XML).
- Captures Tableau fournies dans la conversation « Définir un anti-consultant » et dans sa continuation.
- Script SQL complémentaire : Requete SQL TP2 GARDAIS Romain.sql — 18 blocs de requêtes (création, vérification, historisation, contrôles de doublons et diagnostics horaires).

Note de méthode : les valeurs 1 519, 18 707, 7 335, 11 372 et 49 896 sont recalculées directement à partir du JSON fourni. Les nombres de feuilles et tableaux de bord et la présence des champs/paramètres sont vérifiés dans le

.twbx. Les limites signalées sont conservées pour éviter de présenter comme validées des fonctionnalités non démontrées.

D. Requêtes SQL MySQL — transcription intégrale

Source : « Requete SQL TP2 GARDAIS Romain.sql » transmis par l'auteur. Les instructions ci-dessous sont reprises dans leur ordre d'origine, sans modification de leur logique. Les lignes blanches ont été resserrées pour la lisibilité. Les intitulés et commentaires de lecture sont ajoutés au rapport, non au script.

D.1 Définition des deux tables

DDL : photographie des stations et table d'historique, avec clés et index.

```
USE velibdb;

-- Photo instantanée : 1 ligne par station
CREATE TABLE IF NOT EXISTS station_dispo (
    stationcode           VARCHAR(10) NOT NULL PRIMARY KEY,
    name                  VARCHAR(150) NOT NULL,
    is_installed          ENUM('OUI','NON'),
    capacity              SMALLINT,
    numdocksavailable     SMALLINT,
    numbikesavailable     SMALLINT,
    mechanical            SMALLINT,
    ebike                 SMALLINT,
    is_renting            ENUM('OUI','NON'),
    is_returning          ENUM('OUI','NON'),
    duedate               DATETIME NULL,
    lat                   DECIMAL(10,7),
    lon                   DECIMAL(10,7),
    commune               VARCHAR(100),
    code_insee            CHAR(5),
    station_opening_hours VARCHAR(255) NULL
);
```

```
USE velibdb;

-- Historique : 1 ligne par station et par relevé
CREATE TABLE IF NOT EXISTS station_historique (
    id                BIGINT AUTO_INCREMENT PRIMARY KEY,
    releve_at         DATETIME NOT NULL,
    stationcode       VARCHAR(10) NOT NULL,
    capacity          SMALLINT,
    numdocksavailable SMALLINT,
    numbikesavailable SMALLINT,
    mechanical        SMALLINT,
    ebike             SMALLINT,
    is_renting        ENUM('OUI','NON'),
    INDEX idx_releve (releve_at),
    INDEX idx_station (stationcode)
);
```

D.2 Présence des tables et inspection des données

Requêtes de découverte et de comptage des enregistrements importés.

```
USE velibdb;

SHOW TABLES;
```

```
USE velibdb;

SELECT 'station_dispo' AS nom_table,
       COUNT(*) AS nombre_lignes
FROM station_dispo

UNION ALL

SELECT 'station_historique' AS nom_table,
       COUNT(*) AS nombre_lignes
FROM station_historique;
```

```
USE velibdb;

SELECT
    stationcode,
    name,
    commune,
    capacity,
    numbikesavailable,
    mechanical,
    ebike,
    duedate
FROM station_dispo
LIMIT 20;
```

D.3 Insertion et inspection des relevés historiques

Copie de duedate dans releve_at, dénombrement et consultation des relevés récents.

```
USE velibdb;

INSERT INTO station_historique (
    releve_at,
    stationcode,
    capacity,
    numdocksavailable,
    numbikesavailable,
    mechanical,
    ebike,
    is_renting
)
SELECT
    duedate,
    stationcode,
    capacity,
    numdocksavailable,
    numbikesavailable,
    mechanical,
    ebike,
    is_renting
FROM station_dispo
WHERE duedate IS NOT NULL;


SELECT COUNT(*) AS nombre_releves
FROM station_historique;


USE velibdb;

SELECT
    stationcode,
    releve_at,
    capacity,
    numbikesavailable,
    mechanical,
    ebike
FROM station_historique
ORDER BY releve_at DESC
LIMIT 20;
```

D.4 Détection et prévention des doublons

Recherche des doublons puis contrainte d'unicité sur le couple station/horodatage.

```
USE velibdb;

SELECT
    stationcode,
    releve_at,
    COUNT(*) AS nombre
FROM station_historique
GROUP BY stationcode, releve_at
HAVING COUNT(*) > 1;


USE velibdb;

ALTER TABLE station_historique
ADD CONSTRAINT uq_station_releve
UNIQUE (stationcode, releve_at);
```

D.5 Couverture des dates et des stations

Agrégations par horodatage, bornes temporelles et sélection des anciens relevés.

```
USE velibdb;

SELECT
    DATE_FORMAT(releve_at, '%Y-%m-%d %H:%i:%s') AS date_releve,
    COUNT(*) AS nombre_stations
FROM station_historique
GROUP BY releve_at
ORDER BY releve_at DESC;


USE velibdb;
```

```
SELECT
    COUNT(*) AS total_enregistrements,
    COUNT(DISTINCT stationcode) AS stations_distinctes,
    COUNT(DISTINCT releve_at) AS horodatages_distincts,
    MIN(releve_at) AS premier_releve,
    MAX(releve_at) AS dernier_releve
FROM station_historique;


USE velibdb;

SELECT
    stationcode,
    releve_at,
    capacity,
    numbikesavailable
FROM station_historique
WHERE releve_at < '2026-10-23 00:00:00'
ORDER BY releve_at ASC
LIMIT 20;
```

D.6 Diagnostics par jour et par heure

Tests des plages 08 h/18 h, 09 h/19 h et 12 h/15 h, ainsi que les moyennes et totaux.

```
SELECT
    COUNT(*) AS nombre_lignes,
    COUNT(DISTINCT releve_at) AS nombre_releves,
    MIN(releve_at) AS premier_releve,
    MAX(releve_at) AS dernier_releve,
    COUNT(DISTINCT CASE
        WHEN HOUR(releve_at) = 8 THEN releve_at
    END) AS releves_8h,
    COUNT(DISTINCT CASE
        WHEN HOUR(releve_at) = 18 THEN releve_at
    END) AS releves_18h
FROM velibdb.station_historique;


USE velibdb;

SELECT
    DATE(releve_at) AS date_releve,
    HOUR(releve_at) AS heure,
    COUNT(*) AS nombre_lignes,
    COUNT(DISTINCT stationcode) AS nombre_stations,
    MIN(releve_at) AS premier_horodatage,
    MAX(releve_at) AS dernier_horodatage
FROM station_historique
WHERE HOUR(releve_at) IN (8, 18)
GROUP BY
    DATE(releve_at),
    HOUR(releve_at)
ORDER BY
    date_releve DESC,
    heure ASC;


USE velibdb;

SELECT
    HOUR(releve_at) AS heure,
    COUNT(*) AS nombre_lignes,
    COUNT(DISTINCT stationcode) AS nombre_stations,
    COUNT(DISTINCT releve_at) AS nombre_releves,
    MIN(releve_at) AS premier_releve,
    MAX(releve_at) AS dernier_releve
FROM station_historique
WHERE HOUR(releve_at) IN (9, 19)
GROUP BY HOUR(releve_at)
ORDER BY heure;


SELECT
    DATE(releve_at) AS date_releve,
    COUNT(*) AS nombre_lignes,
    COUNT(DISTINCT releve_at) AS nombre_releves,
    COUNT(DISTINCT stationcode) AS nombre_stations
FROM station_historique
GROUP BY DATE(releve_at)
ORDER BY date_releve;


SELECT
    HOUR(releve_at) AS heure,
    COUNT(*) AS nombre_lignes,
```

```
    COUNT(DISTINCT stationcode) AS nombre_stations,
    ROUND(AVG(numbikesavailable), 2) AS moyenne_velos,
    SUM(numbikesavailable) AS total_velos
FROM station_historique
WHERE DATE(releve_at) = '2026-10-23'
  AND HOUR(releve_at) IN (12, 15)
GROUP BY HOUR(releve_at)
ORDER BY heure;
```

Remarque méthodologique : la transcription d'un script SQL démontre les requêtes préparées et conservées, mais pas leurs résultats d'exécution. Les résultats chiffrés du rapport proviennent des fichiers et des captures contrôlés dans le cadre du TP.